

How Many Lines to Paint the City: Exact Edge-Cover in Temporal Graphs

Advanced Artificial Intelligence - A.L.MA. ¹

Andreas Panayi

Department of Informatics & Telecommunications - NKUA

December 2025



¹andreaspanayi8@gmail.com

Table of Contents

1 Introduction

2 Preliminaries

- Exact Edge-Cover

3 Results

- Algorithm Design
- Cook Reduction

4 Thanks

1. INTRODUCTION

Problem

Informally

In many real-world systems, connections or opportunities (trips, tasks, or contacts) are only available at certain times. The goal is to plan and allocate limited resources (vehicles, crews, robots, agents) so every time-stamped connection is executed.

Formally

Given a dynamic network in which connections are available only at specific times, determine the minimum number of resources required to realise all required connections.

What this models:

- Scheduling vehicles, crews, or agents in transportation and logistics systems.
- Coordinating time-sensitive tasks in multi-agent and robotic systems.
- Planning maintenance, data-collection passes, or service calls that must happen at fixed times.
- Exact-use constraints (each time-stamped task must be performed by exactly one resource).

Practical benefits:

- Reduces operational cost by minimising fleet and crew sizes.
- Improves resource utilisation and decreases idle time.
- Enables robust contingency planning under temporal constraints

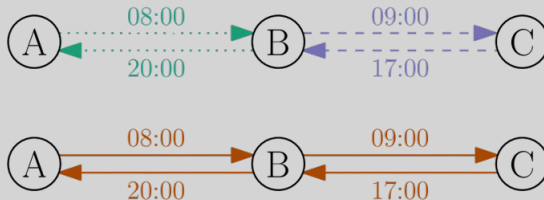
Relation to AI and where it appears

- AI methods (search, combinatorial optimization, scheduling) can compute or approximate minimal-resource allocations for time-dependent tasks.
- *Planning & multi-agent systems*: coordinating many agents under temporal constraints.
- *Predictive + planning pipeline*: data-driven demand prediction (e.g., forecasting future trips or requests) can feed a deterministic scheduler that solves an instance of our problem.
- *Operations research & optimization engines*: specialized solvers or heuristics used in AI/OR for vehicle routing, crew scheduling, dispatching can be applied directly.

- A train operator designs connections to meet demand; next it must choose how many trains to deploy to realise those connections.
- When multiple operators run the network, each sets times independently (service, profit, regulation), increasing scheduling conflict.
- Temporal ordering of connections determines resource reuse: one train can serve multiple connections only if times permit.

Example

- Company X: $A \rightarrow B$ @08:00, $B \rightarrow A$ @20:00 (duration 1h each).
- Company Y: $B \rightarrow C$ @09:00, $C \rightarrow B$ @17:00 (duration 1h each).
- Independently: 2 vehicles needed (one per company).
- Together: one vehicle can serve ($A \rightarrow B$ 08:00) then ($B \rightarrow C$ 09:00) then ($C \rightarrow B$ 17:00) then ($B \rightarrow A$ 20:00).
- Takeaway: temporal ordering enables significant resource savings.



2. MATHEMATICAL MODELING

Graph and Digraph Definitions

Undirected graph (simple)

A *simple graph* $G = (V, E)$ consists of:

a finite vertex set V and an edge set $E \subseteq \binom{V}{2}$.

Each edge is an unordered pair $\{u, v\} \in E$ with $u \neq v$.

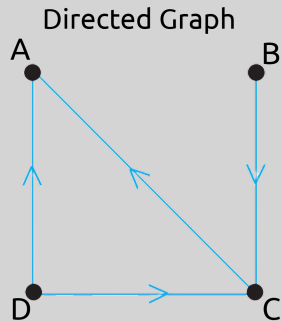
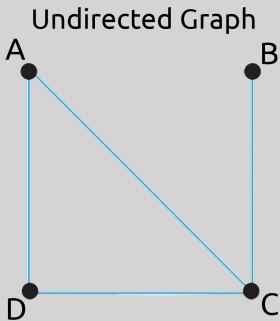
Directed graph (digraph)

A *digraph* $G = (V, E)$ consists of:

a finite vertex set V and an edge set $E \subseteq V \times V$.

Each ordered pair $(u, v) \in E$ is a directed edge from u to v where $u \neq v$.

Example of Graph/Digraph



Temporal graph — definition

Temporal graph (formal definition)

A *temporal graph* is a pair (G, λ) where $G = (V, E)$ is either a simple graph or a directed graph (digraph) and

$$\lambda : E \rightarrow 2^{\mathbb{N}}$$

is a labeling function that assigns to every edge a (finite) set of time-instants. For an edge $e \in E$, $\lambda(e) \subseteq \mathbb{N}$ is the set of times at which e is present (available).

Equivalent Definition

For each time $t \in \mathbb{N}$ define the static time-slice

$$G_t = (V, E_t), \quad E_t = \{e \in E : t \in \lambda(e)\}.$$

Hence the temporal graph (G, λ) can be seen as a sequence $(G_t)_{t \in \mathbb{N}}$ of static graphs.

Temporal Graph: Alternative Definition

Simpler Definition

A *temporal graph* $G := \langle G, \mathcal{E} \rangle$ is defined by:

- an underlying static graph $G = (V, E)$, and
- a sequence of edge-sets $\mathcal{E} = (E_1, E_2, \dots, E_{t_{\max}})$ where each $E_t \subseteq E$ represents the edges available at time t ,
- with $E = E_1 \cup E_2 \cup \dots \cup E_{t_{\max}}$.

The temporal graph can be equivalently seen as a sequence of static graphs $(G_1, G_2, \dots, G_{t_{\max}})$ with $G_t = (V, E_t)$.

Temporal Graph Example

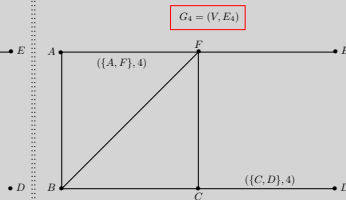
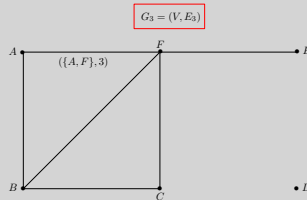
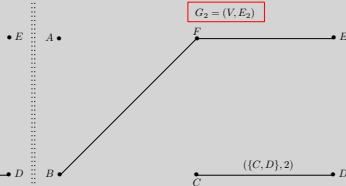
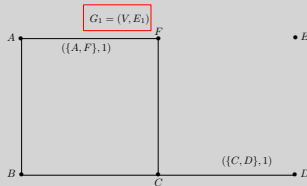
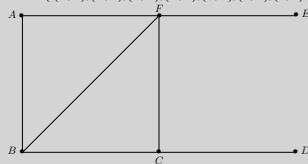
$$G_{temp} = (G, \mathcal{E})$$

$$\mathcal{E} = (E_1, E_2, E_3, E_4)$$

$$G = (V, E)$$

$$V = \{A, B, C, D, E, F\}$$

$$E = \{ \{A, B\}, \{B, C\}, \{C, D\}, \{B, F\}, \{C, F\}, \{F, E\}, \{F, A\} \}$$



Temporal journeys

- A *temporal journey* is a way to travel through the graph over time, using only edges that are present when we traverse them.
- Consecutive edges must respect time:
 - non-strict*: $t_{i+1} \geq t_i$,
 - strict*: $t_{i+1} > t_i$.
- Journeys model feasible movements of a single resource (vehicle, agent) through the dynamic network.

Temporal walks, trails, and paths

Consider a sequence

$$v_0 \xrightarrow[(t_1)]{e_1} v_1 \xrightarrow[(t_2)]{e_2} \cdots \xrightarrow[(t_\ell)]{e_\ell} v_\ell$$

where each $e_i = \{v_{i-1}, v_i\}$ (or (v_{i-1}, v_i)) and $t_i \in \lambda(e_i)$ with $t_1 \leq t_2 \leq \cdots \leq t_\ell$ (or strictly increasing in the strict case).

- *Temporal walk*: vertices and edges may repeat.
- *Temporal trail*: edges are all distinct; vertices may repeat.
- *Temporal path*: all vertices (hence edges) are distinct.

(Non-)Strict Temporal Φ Exact Edge-Cover

Definition: Temporal exact edge-cover

Let $G = \langle (V, E), \mathcal{E} \rangle$ be a temporal graph (lifetime $t_{\max}$) and let $\Phi \in \{\text{path, trail, walk}\}$.

A collection $\mathcal{J} = \{J_1, \dots, J_k\}$ of temporal journeys of type Φ is a *temporal exact edge-cover* of G if every temporal edge $(e, t) \in \mathcal{E}$ appears in exactly one journey $J_i \in \mathcal{J}$.

Problem: (Non-)Strict Temporal Φ Exact Edge-Cover

Input: A temporal graph $G = \langle (V, E), \mathcal{E} \rangle$ and an integer $k \in \mathbb{N}$.

Question: Does there exist a temporal exact edge-cover of G by at most k journeys of type Φ , where each journey is non-strict or strict.

3. SIGNLE JOURNEY

Complexity Results: for $k = 1$

		Path	Trail	Walk
Directed	Strict	linear (P)	linear (P)	linear (P)
	Non-Strict	linear (P)	linear (P)	linear (P)
Undirected	Strict	linear (P)	linear (P)	linear (P)
	Non-Strict	linear (P)	linear (P)	linear (P)

All variants for $k = 1$ are in P.

Strict variant for $k = 1$ — Directed case

Greedy Algorithm:

1. *Feasibility remark:* in any feasible strict $k = 1$ instance each time t satisfies $|E_t| \leq 1$. If some t has two temporal edges the instance is infeasible.
2. Let $(e, t_{\min})$ be the earliest temporal edge (unique under feasibility). In the directed setting the orientation of e fixes the start-terminal.
3. *Greedy construction:* scan temporal edges in increasing time. Maintain the current endpoint v and previous time t_{prev} .
For the next temporal edge (f, t) , append it iff $t > t_{\text{prev}}$ and f is adjacent in the correct direction to v ; update $v \leftarrow$ the new endpoint and $t_{\text{prev}} \leftarrow t$.
If an edge cannot be appended, the greedy attempt fails.
4. *Decision:* accept if the greedy pass produces one journey that covers every temporal edge exactly once; otherwise reject.
 - *Complexity:* each temporal edge is processed once — overall linear time (up to representation), i.e. $O(|V| + |\mathcal{E}|)$.

Strict variant for $k = 1$ — Undirected case

Modified Previous Algorithm:

1. *Feasibility remark:* as above, any feasible strict $k = 1$ instance has $|E_t| \leq 1$ for every t .
2. Let $(e, t_{\min})$ be the earliest temporal edge. Unlike the directed case, the start-terminal is not fixed: there are two possible start endpoints for e .
3. *Two attempts:* for each endpoint u of e do:
 Initialize the journey at u using $(e, t_{\min})$.
 Scan temporal edges in increasing time and greedily append each next edge if it is adjacent to the current endpoint (and time increases).
 If the scan succeeds and covers all temporal edges, accept.
4. *Decision:* accept if at least one endpoint guess yields a full cover; otherwise reject.
 - *Complexity:* two greedy passes (constant factor) — overall linear time $O(|V| + |\mathcal{E}|)$.

Informal reductions — definitions

- **Turing / Cook (algorithmic) reduction:** an algorithm for A that runs in polynomial time while making polynomially many calls to a black-box solver for B . Think: “call a B -subroutine repeatedly and combine the results”.
- **Karp / many-one reduction:** a polynomial-time computable function f that maps instances x of A to instances $f(x)$ of B such that x is a yes-instance of A iff $f(x)$ is a yes-instance of B .

Use of reduction

- **Cook reduction:** If we solve A by a polynomial-time algorithm that makes only polynomially many calls to a solver for B , then any polynomial-time algorithm for B immediately yields a polynomial-time algorithm for A .
- **Karp reduction for membership:** if there is a polynomial-time many-one reduction $A \leq_p B$ and B has a polynomial-time algorithm, then we can decide A in polynomial time.
- **Karp reduction for hardness:** By giving a polynomial-time many-one reduction $A \leq_p B$ where A is NP-hard we conclude B is NP-hard. If additionally $B \in \text{NP}$, it is NP-complete.

Eulerian path and cycle

Definition: Eulerian Paths/Trail

- **Eulerian trail / path:** a trail that uses every edge of the graph exactly once.
- **Eulerian cycle / circuit:** an Eulerian trail that starts and ends at the same vertex.

We know that:

1. There exists a polynomial-time algorithm that decides whether a given graph (directed or undirected) admits an Eulerian cycle or path, and that identifies the possible start/end vertices when a trail (non-closed) exists.
2. The algorithm runs in linear time $O(|V| + |E|)$ in the standard adjacency representation (e.g., modified Hierholzer's algorithm).

Non strict variant for $k = 1$

Core insight

Process the temporal graph one time-slice at a time and decide, for each slice, which local traversals are feasible; then check whether those local choices can be chained so that the end vertex of slice t equals the start vertex of slice $t + 1$. This slice-by-slice chaining encapsulates the temporal feasibility of a single global journey that uses every temporal edge exactly once.

- Applies to both **directed** and **undirected** graphs.
- Applies to all journey types $\Phi \in \{\text{path, trail, walk}\}$; only the *local feasibility test* per slice changes with Φ .

Algorithm for Non Strict $k = 1$

1. **For each time-slice** $t = 1, \dots, t_{\max}$: compute the set P_t of feasible $(start, end)$ pairs for a traversal that uses every edge in E_t exactly once, under the chosen journey type Φ and orientation (directed/undirected).
2. **Dynamic chaining**: propagate reachability of endpoints across slices.

Initialise $S_1 = \{e_1 : (s_1, e_1) \in P_1\}$ (or restrict by fixed start terminal if given).

For $t = 2, \dots, t_{\max}$ update

$$S_t = \{e_t \mid (s_t, e_t) \in P_t \text{ and } s_t \in S_{t-1}\}.$$

3. **Decide**: accept iff $S_{t_{\max}}$ is non-empty (there exists a compatible sequence of (s_t, e_t) giving one global journey that covers every temporal edge).
- **Complexity remark**: each slice is processed once and chaining is a simple propagation — overall polynomial time.

4. TWO JOURNEYS

Complexity Results for $k = 2$

		Path	Trail	Walk
Directed	Strict	linear (P)	linear (P)	linear (P)
	Non-Strict	linear (P)	NP-complete	linear (P)
Undirected	Strict	linear (P)	linear (P)	linear (P)
	Non-Strict	linear (P)	NP-complete	linear (P)

$k = 2$ Directed Strict Temporal Path

We show that the $k = 2$ strict directed temporal-path exact edge-cover can be decided by a Karp reduction to an XOR(2) formula.

Reduction idea (very compact)

- Sweep edges in increasing time, maintaining two prefix paths with endpoints and visited sets.
- Attach forced edges deterministically.
- At an ambiguous hub (both prefixes can take an edge) create a Boolean variable recording which path takes it.
- Emit binary clauses whenever a combination of choices would cause a revisit or impossible attachment.
- The resulting XOR(2)SAT formula is satisfiable iff two valid strict temporal paths exist.

Similar Polynomial Cases via the Same Idea

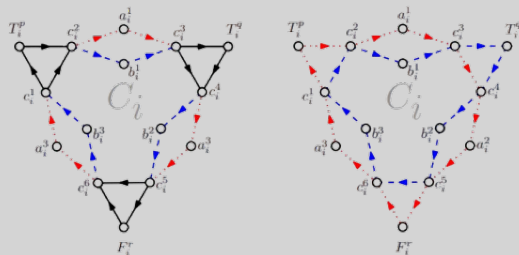
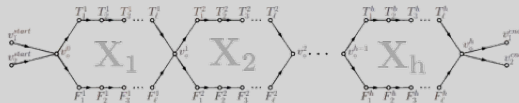
The same sweep-and-local-choice reduction (introduce Boolean variables at ambiguous hubs, emit binary constraints to forbid revisits) can be adapted with small changes (adjusting local feasibility checks for paths vs. trails and respecting strict vs. non-strict timing) to yield polynomial-time algorithms for the following $k = 2$ variants:

- strict temporal *trails* on directed temporal graphs.
- non-strict temporal *paths* on directed temporal graphs.
- non-strict temporal *trails* on directed temporal graphs.
- strict and non-strict temporal *paths/trails* on undirected temporal graphs.

$k = 2$ Non-strict Temporal Trail: NP-C

The non-strict temporal trail exact edge-cover problem with $k = 2$ temporal trails is *NP-complete* on both directed and undirected temporal graphs.

Proof idea. The hardness is shown via a polynomial-time reduction from a classical NP-complete problem called NAE(3)SAT.



5. MULTIPLE JOURNEYS

Complexity of Problem when $k \geq 3$

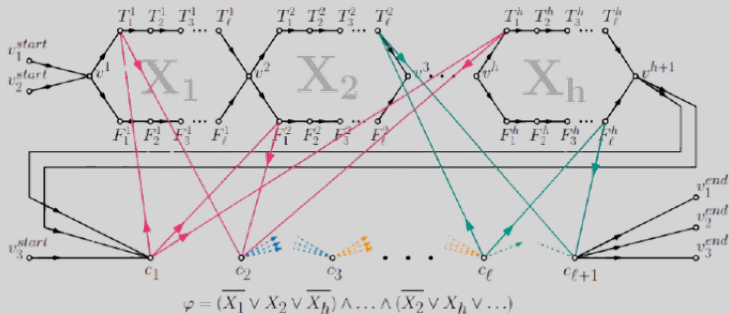
		Path	Trail	Walk
Directed	Strict	NP-complete	NP-complete	P
	Non-Strict	NP-complete	NP-complete	W[2]-Hard
Undirected	Strict	NP-complete	NP-complete	NP-C & XP
	Non-Strict	NP-complete	NP-complete	NP-C & XP

- **Computational hardness (decision):**
 - We will prove NP-completeness for the relevant variants.
 - Proof technique: polynomial-time many-one (Karp) reductions from standard NP-complete problems.
- **Approximation hardness (optimisation):**
 - We will show strong inapproximability for paths and trails.
 - Proof technique: gap-introducing reductions (no polynomial-time α -approximation for any $\alpha > 1$).

Strict Temporal Path Directed: NP-C

The strict temporal path exact edge-cover problem on directed temporal graphs is *NP-hard*.

Proof idea. The hardness is shown by a polynomial-time (Karp) reduction from $NAE(k)SAT$.



From our previous results we deduce that

- the non-strict temporal path exact edge-cover on directed temporal graphs is NP-hard (hence NP-complete);
- the strict temporal path exact edge-cover on undirected temporal graphs is NP-hard (hence NP-complete);
- the non-strict temporal path exact edge-cover on undirected temporal graphs is NP-hard (hence NP-complete).

Inapproximability — Paths and Trails

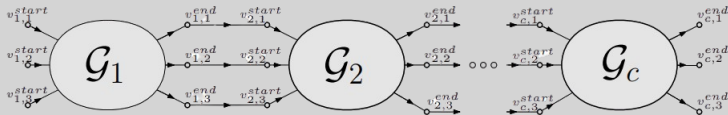
- For every $\alpha > 1$, there is no polynomial-time α -approximation algorithm for the (non-)strict temporal path exact edge-cover problem on directed or undirected temporal graphs, unless $P = NP$.
- For every $\alpha > 1$, there is no polynomial-time α -approximation algorithm for the (non-)strict temporal trail exact edge-cover problem on directed or undirected temporal graphs, unless $P = NP$.

Hardness of Approximation: Proof Idea

For every $\alpha > 1$, there is no polynomial time α -approximation algorithm for (non-)strict temporal path on (un)idirected temporal graphs unless $P = NP$.

Proof Technique:

The inapproximability of temporal path and trail edge-cover problems is shown using a *gap-introducing reduction*



Corollaries for Other Variants

- Both the strict and non-strict temporal trail exact edge-cover problems are *NP-complete* on *directed and undirected* temporal graphs.
- For any $\alpha > 1$, there is *no polynomial-time α -approximation algorithm* for strict or non-strict temporal trail exact edge-cover on *directed or undirected* temporal graphs, unless $P = NP$.

Walks: Computational Results

- *Directed, strict*: polynomial-time solvable.
- *Directed, non-strict*: $W[2]$ -complete (parameterized hardness in k).
- *Undirected (strict and non-strict)*: NP-complete.

Strict Temporal Walk Directed: Polynomial Algorithm

Input: edgestream of a directed temporal graph (edges in increasing time order).

Greedy - Streaming algorithm

```
1: For all  $x \in V$ :  $c[x] \leftarrow 0$            ▷ number of walks currently ending at  $x$ 
2:  $starts \leftarrow 0$ 
3: for each temporal edge  $(u \rightarrow v, t)$  in increasing  $t$  do
4:   if  $c[u] > 0$  then
5:      $c[u] \leftarrow c[u] - 1$            ▷ extend an existing walk ending at  $u$ 
6:   else
7:      $starts \leftarrow starts + 1$            ▷ start a new walk at  $u$ 
8:   end if
9:    $c[v] \leftarrow c[v] + 1$            ▷ the walk now ends at  $v$ 
10: end for
```

The number of walks equals starts; actual walks can be recovered by tracking which walk is extended at each step.

Non-Strict Temporal Walk Directed: W[2]-Hard

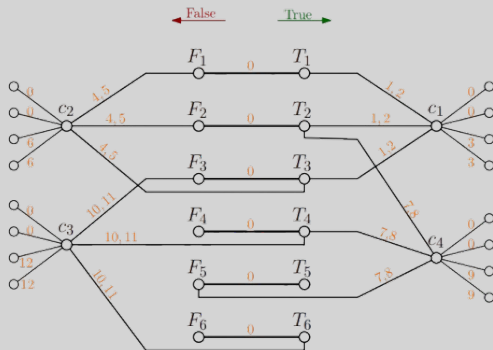
To prove that the temporal exact edge-cover problem with non-strict walks on directed temporal graphs is NP-complete and W[2]-hard (when parameterized by the number of walks), we use a reduction from the *Hitting Set* problem.

Proof. Given a *Hitting Set* instance $(N = \{1, \dots, n\}, \mathcal{S} = \{S_1, \dots, S_m\}, k)$, assume that all sets have size 2 or larger, otherwise augment them with auxiliary elements. We construct a temporal graph $G = \langle (V, E), \mathcal{E} \rangle$ with $V = N$. For every set $S_j = \{x_1, \dots, x_{|S_j|}\}$, we construct a cycle between its elements where all edges have time label j , i.e., for each $0 < i < |S_j|$ we add $((x_i, x_{i+1}), j)$, and lastly $((x_{|S_j|}, x_1), j)$.

Strict Temporal Walk Undirected: NP-C

The strict temporal walk exact edge-cover problem on undirected temporal graphs is *NP-hard*.

Proof idea: The hardness is shown by a polynomial-time reduction from 3-SAT.

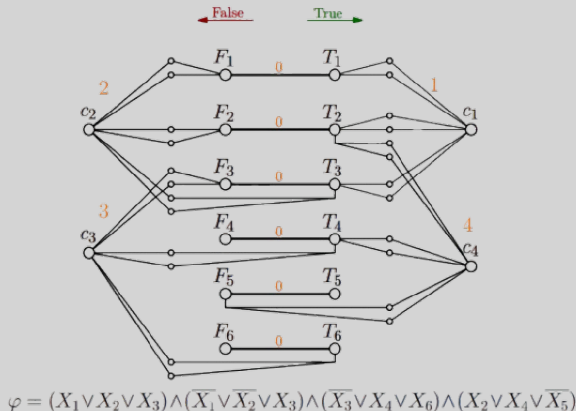


$$\varphi = (X_1 \vee X_2 \vee X_3) \wedge (\overline{X_1} \vee \overline{X_2} \vee X_3) \wedge (\overline{X_3} \vee X_4 \vee X_6) \wedge (X_2 \vee X_4 \vee \overline{X_5})$$

Non-strict Temporal Walk Undirected: NP-C

The non-strict temporal walk exact edge-cover problem on undirected temporal graphs is *NP-hard*.

Proof idea: The hardness is shown by a polynomial-time (Karp) reduction from 3-SAT.



Acknowledgments

Thanks for you attention 😊